

Verifier-Grounded Evaluation of LLM-Generated Network Configurations: A Preregistered NetConfig-Semantic Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered, artifact-anchored paired experiment (CAND-2026-001-NetConfig-Semantic-v1; preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdebb71a37) comparing a deterministic template baseline to a single-pass LLM generator (declared_model_version = gpt-5-mini) on $N = 120$ synthetic intent→configuration tasks using a deterministic validator. Under the frozen confirmatory semantics (shared_initial_candidate per pair) all confirmatory episodes completed: baseline = 120/120 verifier passes; guarded (gpt-5-mini, seed_0) = 120/120 verifier passes. Paired difference = 0.00; 95% bootstrap percentile CI [0.00, 0.00] (10,000 resamples, seed = 1729); exact McNemar two-sided $p = 1.0$. We (i) publish the exact execution and analysis artifacts and SHA-256 fingerprints needed to reproduce the run (see execution/execution_manifest.json in the artifact bundle), (ii) diagnose—using archived artifacts—why the paired outcome is uninformative about independent generative reliability (preregistered shared_initial_candidate design, template-tight task synthesis, and validator–task coupling), and (iii) provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory design, validator diversity, and expanded task heterogeneity) that preserve reproducibility while producing informative independent comparisons. The immutable initial master prompt is archived (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdff1582bb39b15ba).

I. INTRODUCTION

Motivation. Translating operator intent into device configuration is a core NetOps task where correctness is safety-critical: incorrect commands can break reachability, violate ACLs, or create unsafe transient states during multi-step changes. Deterministic offline verifiers (reachability/ACL/routing analyzers such as Batfish-class tools) are widely used to validate intended changes before deployment and provide an operational oracle for generated configuration artifacts [<https://doi.org/10.1145/3603269.3604866>]. Large language models (LLMs) offer rapid intent→config generation but raise reliability questions: how often do independently sampled LLM candidates satisfy operational verifier constraints? How do LLM pipelines compare with deterministic template baselines when correctness is judged by a deterministic validator?

Research question and contribution. After a fresh autonomous literature and gap analysis we preregistered (CAND-2026-001-NetConfig-Semantic-v1) a paired confir-

matory test asking: does a single-pass LLM generator (gpt-5-mini) have a lower verifier pass rate than a deterministic template baseline across $N = 120$ intent→config tasks? Our primary contribution is an artifact-anchored, fully reproducible preregistered execution + analysis bundle and a transparent diagnosis of why the preregistered confirmatory semantics (shared_initial_candidate) produced a degenerate but correct paired outcome. We (1) publish the exact artifacts and SHA-256 fingerprints required for byte-for-byte reruns; (2) report confirmatory counts and preregistered statistical inferences grounded in the archived traces (baseline = 120/120, guarded = 120/120; paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar $p = 1.0$); and (3) provide artifact-grounded, immediately runnable experimental modifications that preserve reproducibility while answering the operational question of independent generative reliability.

II. RELATED WORK

Verifier-grounded NetOps and intent→configuration. Offline semantics analyzers that compute network final and transient state from device configurations are established engineering controls in pre-deployment validation. Batfish and related tools demonstrate how high-fidelity configuration analysis enables operators to detect semantic errors before changes reach production; Batfish’s engineering lessons emphasize scalability, fidelity, and the operational value of deterministic verification in NetOps workflows [10.1145/3603269.3604866]. These verifiers produce machine-checkable oracles (reachability matrices, ACL equivalence, route-preservation checks and transient-state invariants) that directly map to concrete operational failure modes (lost reachability, unintended route leaks, unsafe transient downtimes).

LLM→NetOps evaluations and generator–validator patterns. Recent work on intent-driven configuration generation and related pipelines has converged on generate–validate–repair architectural patterns: an LLM proposes candidate configurations, a deterministic verifier checks semantics, and a repair loop attempts to fix validator failures. Surveys and position papers on LLM-based network management identify this pattern and emphasize that deterministic validators are necessary to move beyond syntactic or fluency-based evaluation metrics [10.1002/nem.70029]. Empirical studies of LLM repair loops in adjacent domains show diminishing returns from unbounded iteration and highlight that evaluation out-

comes depend strongly on orchestration and feedback design rather than model identity alone; these findings motivate careful treatment of repair budgets and sampling semantics when designing experiments that measure operational reliability.

Evaluation pitfalls: semantic scoring vs. generative variability. Prior evaluations have sometimes conflated different scientific questions: (i) how often a particular single candidate produced by a pipeline passes a verifier (a candidate-level success), (ii) how reliably independently sampled candidates satisfy verifier constraints (a generative-reliability probability), and (iii) whether a repair loop can raise a failed candidate to pass. Studies that report per-candidate pass rates without specifying generation semantics or seed treatment risk ambiguity. Our preregistered design explicitly fixed generation semantics (shared_initial_candidate) and thus answered a narrowly specified estimand: whether a deterministic verifier treats identical artifacts differently across scoring conditions—an important but distinct question from independent LLM reliability. The distinction maps directly onto operational decisions: operators choosing to accept a single canonical LLM candidate with verification differ from operators who rely on repeated sampling or repair loops to achieve probabilistic guarantees.

Validator–task coupling and construct validity. When tasks are programmatically templated and validators enforce the very workflow_policies encoded in those templates, canonical candidates that mirror the template structure will tend to pass deterministically. This validator–task coupling can inflate pass rates relative to open distributions of natural intents. The literature on intent-driven generators recognizes this trade-off: narrow, template-aligned tasks allow reproducible, deterministic evaluation useful for principled method comparisons, but they limit external validity to more heterogeneous operational intents (surveyed in [10.1002/nem.70029]).

Preregistration, artifact release, and reproducibility. A complementary body of work emphasizes transparent artifact publication (model traces, seeds, verifier logs) and preregistration to make explicit what a given test measures and to permit deterministic reruns under alternative semantics. Our contribution adopts this practice: we preregistered the estimand, generation semantics, and analysis plan, and we release the full per-episode scoring traces and SHA-256 manifest so that others can reexecute the experiment under different sampling semantics (for example, independent_per_condition sampling or multi-seed confirmatory designs) while preserving deterministic reproducibility of each variant. In net effect, verifier grounding plus exhaustive artifact publication clarifies both the scientific question answered and the minimal modifications required to answer adjacent operational questions.

III. METHODOLOGY

Preregistration and frozen design. The full preregistration (CAND-2026-001-NetConfig-Semantic-v1) was archived prior to any model calls (SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fde71a37e). The preregistered confirmatory design specified a paired comparison across $N = 120$ tasks (40 tasks per topology

class: edge, datacenter, multi-AS), the primary estimand paired_success_rate_difference_guarded_minus_baseline, confirmatory generation_semantics = "shared_initial_candidate", one canonical seed_0 per pair for the confirmatory analysis, the deterministic validator deterministic_netops_validator_v5 (validator_version 5.0) and the scoring rule full_intent_constraint_validation. The preregistered analysis executor was paired_binary_analysis_v1 with bootstrap inference (10,000 resamples, seed = 1729). The preregistration and analysis plan (including failure treatment, retry policy, and contamination heuristics) are published in the artifact bundle.

Task synthesis and templates. Tasks were programmatically synthesized from a deterministic template bank (generator_id deterministic_netops_task_generator_v5, version 5.0). Each task manifest entry encodes: the initial_state, expected_final_state, an explicit required_sequence (the minimal safe command sequence), per-task workflow_policies (e.g., must_be_down_for_fields, minimum_active_in_groups), allowed_touched_fields, and a difficulty annotation (changed_interface_count, transient_rule_count). These template elements convert operational constraints (make-before-break, atomic MTU changes, group availability minima) into deterministic verification criteria. Representative task manifests and their exact file locations are archived (execution/task_manifest.jsonl; see artifact_examples within the bundle).

Model, orchestration, and exact generation semantics. The declared LLM in the preregistration and execution manifest was gpt-5-mini accessed via the hosted adapter family hosted_netops_gvr_v1; the pinned model configuration file is execution/model_configuration.json (SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82). Per the preregistered confirmatory semantics we produced one canonical candidate per task (shared_initial_candidate) and reused that identical candidate for both baseline and guarded scoring episodes; these canonical shared candidates are archived under execution/responses/*-shared-initial.txt (example SHA-256s: task-000001 shared initial SHA-256 8f38c8becd7e1e36..., task-000002 SHA-256 ae967f70dfb6ccb8..., task-000003 SHA-256 f7f4db249ecbbce...). The execution manifest records that model_calls_used = 120 while planned_episode_count = 240 (the shared_initial design reduces distinct generation calls per condition and is explicitly documented in the manifest).

Deterministic validation and scoring. Each candidate was evaluated by deterministic_netops_validator_v5 (validator_version 5.0). The validator pipeline performs: normalization and parsing of candidate commands; enforcement of per-task workflow_policies and transient constraints (e.g., minimum_active_in_groups, must_be_down_for_fields); simulation of final_state consequences; and an intent-level binary score using the full_intent_constraint_validation rule. Redeployment. JSON scoring traces (execution/scoring/*_json) include parsed_command_count, required_command_count, normalized_configuration, validation_before/after final_state

dictionaries, and violation lists. Representative scoring files are archived (e.g., execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...).

Execution accounting, retries, contamination heuristics. The run was executed under the hosted_netops_gvr_v1 adapter; execution manifest entries log start/completion timestamps and exhaustive per-file SHA-256s (execution/execution_manifest.json). Planned_episode_count = 240; completed_episode_count = 240; model_calls_used = 120 (shared_initial generation per pair counted once); failed_episode_count = 0. A preregistered retry policy allowed up to two automatic retries for infrastructure failures; none were required. Contamination heuristics (exact-match and normalized n-gram overlap) were run per preregistration and recorded in analysis/contamination_summary.csv; no confirmatory items were flagged under the chosen thresholds (however the preregistration and Disclosure explicitly note that absence of a flag is not proof of zero pretraining exposure).

Primary inferential procedure. For each pair i ($i = 1..120$) the baseline_i and guarded_i binary verifier outcomes were recorded. The paired estimator is $\text{mean}_{\{i\}}(\text{guarded}_i - \text{baseline}_i)$. Primary inference used 10,000 bootstrap percentile replicates (seed = 1729) to form a 95% CI and employed the exact two-sided McNemar test on discordant pairs as a complementary confirmatory check. All analysis code, bootstrap parameters, and the analysis log are archived under analysis/ (see analysis/analysis_log.jsonl SHA-256 ff20bc07...).

Key artifact index (representative, for rapid audit). The following canonical artifact paths and SHA-256 fingerprints are included in the submission bundle to enable byte-for-byte reruns and immediate inspection:

- preregistration: provenance/preregistration_CAND-2026-001.json (SHA-256 7db1663cf3982c8e...)
- initial master prompt: provenance/master_prompt.txt (SHA-256 1872df1e1805d2d9...)
- model configuration: execution/model_configuration.json (SHA-256 a40e96e212ab87da...)
- execution manifest (full artifact index): execution/execution_manifest.json
- shared canonical candidate (example): execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...)
- scoring trace (example): execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...)
- analysis artifacts: analysis/condition_summary.csv (SHA-256 9c77c96f37c4b6ff...), analysis/paired_contingency_table.csv (SHA-256 9f25790c7eeaafb3...).

Deviations and transparency. The methodology above follows the frozen preregistration; all deviations, retries, exclusions, and per-file SHA-256 fingerprints are recorded in the execution manifest and analysis logs. The artifact bundle provides the exact provenance needed for external auditors to re-execute the analysis under alternative generation semantics (e.g., independent_per_condition or multi-seed confirmatory designs) without introducing unspecified choices.

IV. RESULTS

Confirmatory outcome and exact, archived counts. All preregistered confirmatory episodes completed and are fully archived in the execution manifest (execution/execution_manifest.json). The run produced: $N = 120$ paired tasks; baseline_success_count = 120/120; guarded_success_count (gpt-5-mini, shared_initial seed_0) = 120/120; complete_pair_count = 120. The paired contingency table is therefore degenerate: $n_{11} = 120$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$ (analysis/paired_contingency_table.csv; SHA-256 9f25790c7eeaafb3562e0710e0cf10b7a47a55ea00fcbcf02eee324f79afafe7). Model call accounting recorded model_calls_used = 120 and maximum_model_calls = 240 (execution/execution_manifest.json).

Confirmatory statistical inference (preregistered). Following the frozen analysis plan (paired_binary_analysis_v1) we computed the primary estimator (mean of guarded_i - baseline_i over the 120 pairs) = 0.00. A 10,000-resample percentile bootstrap (seed = 1729 as preregistered) produced a 95% CI [0.00, 0.00]. The exact McNemar two-sided test on the paired 2×2 table yields $p = 1.0$. All analysis artifacts (condition_summary.csv SHA-256 9c77c96f37c4b6ff..., paired_difference_figure.svg SHA-256 ae5b8e0c644f8cf7...) are included in the bundle and match these reported values.

Representative validator traces and command counts (artifact grounding). Each scored episode includes a deterministic validator trace showing parsed_command_count, required_command_count, normalized_configuration, and structured before/after final_state. Example: execution/scoring/task-000001-guarded.json (SHA-256 0d56c1add7c5a57f...) records parsed_command_count = 4, required_command_count = 4, and validation_after.valid = true; the canonical candidate used for the pair is archived at execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...). Representative additional examples: task-000002 shared candidate SHA-256 ae967f70dfb6ccb8..., task-000003 shared candidate SHA-256 f7f4db249ecbbce.... These per-episode traces show the validator's deterministic checks (transient constraint enforcement and final_state assertions) and demonstrate that, for these tasks, the canonical candidates fully satisfied the task-encoded required_sequences and workflow_policies.

Why the confirmatory contingency is degenerate — artifact-visible causal chain. The degenerate paired outcome follows directly from two preregistered, archived design choices visible in the bundle: (1) confirmatory generation_semantics = "shared_initial_candidate" (execution/responses/*-shared-initial.txt) — a single canonical LLM candidate was generated per task and reused verbatim for both baseline and guarded scoring episodes; and (2) task templates encode explicit required_sequences and workflow_policies that are satisfied by canonical, template-aligned candidates. Because the validator is deterministic, identical inputs necessarily yield identical pass/fail outputs. The execution and scoring artifacts (execution/responses/* and execution/scoring/*) provide a

precise, auditable record that links the reused canonical candidate to identical validator outcomes across both conditions for each pair.

Contamination checks and reproducibility notes. Preregistered contamination heuristics (exact-match and normalized n-gram overlap) flagged no confirmatory items under the selected thresholds; the recorded contamination summary is archived at `analysis/contamination_summary.csv`. We explicitly note that absence of a heuristic flag is not a formal proof of zero pretraining exposure. To enable independent audit, all provider call traces and `call_cache` artifacts are published (`execution/provider_calls/*.json`, `execution/call_cache/*`); the full manifest (`execution/execution_manifest.json`) contains per-file SHA-256 fingerprints to permit byte-for-byte verification (example: `execution/model_configuration.json` SHA-256 `a40e96e212ab87da...`).

What these results do and do not show. By construction the confirmatory test correctly answers its preregistered estimand—whether the deterministic verifier treats identical canonical candidates differently under two scoring conditions; the answer is no (paired difference = 0). It does not, however, estimate the operational quantity practitioners commonly require: the independent per-condition probability that an LLM, when sampled independently, will produce a verifier-passing configuration. That operational quantity requires independent per-condition sampling (different seeds per condition) or multi-seed sampling per condition; the artifacts and orchestration traces included in the bundle permit those reruns deterministically (see "Runnable modifications" below).

Runnable, artifact-grounded next steps. The archived pipeline supports immediately reproducible reruns that answer the independent generative reliability question while preserving preregistration and determinism: (A) `independent_per_condition` confirmatory rerun — generate a distinct seed per condition per task (e.g., `baseline seed_0`, `guarded seed_1`), rescore with `deterministic_netops_validator_v5`, and compute paired contrasts; (B) multi-seed confirmatory design — draw $K \geq 3$ independent seeds per condition per task and estimate per-condition pass probabilities with bootstrap CIs; (C) validator diversification — run a second independent verifier and report intersection/union pass rates to reduce validator–task coupling. Because every element (task generator, model configuration, provider calls, and validator) is archived with SHA-256 fingerprints, these reruns yield deterministic, auditable outputs suitable for confirmatory inference.

V. DISCUSSION

Expanded, artifact-grounded interpretation and actionable diagnostics.

Precise inferential scope. The confirmatory analysis correctly answers the preregistered estimand as written: under `shared_initial_candidate` semantics, does the validator produce different pass/fail outcomes between two labeled scoring episodes? The archived contingency (`analysis/paired_contingency_table.csv` SHA-256 `9f25790c7eeafb3...`) shows a logically inevitable answer of

"no difference" because the input artifact was identical per pair. Readers must therefore interpret the reported estimator (paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar $p = 1.0$) as a validation of the execution/analysis pipeline's determinism under identical inputs—not as evidence that the LLM would produce verifier-valid outputs under independent sampling in realistic deployments.

Artifact evidence chain that produced degeneracy. Three artifact classes in the bundle make the causal chain auditable: (1) canonical candidate files (`execution/responses/*-shared-initial.txt`; e.g., `task-000001` SHA-256 `8f38c8becd7e1e36...`, `task-000002` SHA-256 `ae967f70dfb6c...`), (2) per-episode scoring traces that record parsing and validation outcomes (`execution/scoring/*.json`; e.g., `task-000001-guarded.json` SHA-256 `0d56c1add7c5a57f...`), and (3) the paired contingency and condition summaries (`analysis/paired_contingency_table.csv` SHA-256 `9f25790c...`, `analysis/condition_summary.csv` SHA-256 `9c77c96f37c4b6ff...`). These three artifact types form a deterministic provenance path: `shared_initial` candidate → deterministic validator → identical binary score. The bundle contains the exact SHA-256 for each artifact enabling byte-for-byte verification of this chain without re-running the model (`execution/execution_manifest.json` lists the full manifest).

Why this matters for operational evaluation. Operational decision makers care about independent generative reliability: the probability that an LLM sampled anew (or an LLM ensemble, or an agentic pipeline) will produce a verifier-valid change under realistic prompt/seed/temperature variability and natural-language intent diversity. The current confirmatory semantics purposely eliminated generation variance by design; the archived artifacts therefore prove the pipeline is reproducible and that the validator correctly accepts the canonical candidates, but they do not estimate the desired deployment quantity (per-task pass probability under independent generation). The distinction is subtle but consequential: reproducible deterministic scoring is necessary for trustworthy pipelines, but it is not sufficient to establish generative robustness.

Runnable, artifact-grounded experiment modifications (preserving reproducibility). Using only the supplied artifacts and configuration files one can deterministically produce informative reruns that estimate independent generative reliability without changing validator code or the template bank: (A) `independent_per_condition` confirmatory rerun — for each pair generate two independent candidates (one per condition) using distinct seeded calls to the archived generator (the generator id and configuration are archived at `execution/model_configuration.json` SHA-256 `a40e96e2...`), then rescore producing a nondegenerate paired table; (B) multi-seed confirmatory design — for each condition draw $K \geq 3$ independent seeds per task and compute per-condition pass probabilities and their bootstrap CIs (this was included as a preregistered exploratory plan); (C) validator diversification — add a second verifier (for example, an independent reachability or ACL analyzer and report intersection

and union pass rates) to reduce estimator sensitivity to any single validator’s interpretation of workflow_policies. Each of these modifications is fully implementable using only the published code, seeds, and provider-call traces: because execution/provider_calls/*.json and execution/call_cache/* are included, external auditors can replay, re-seed, and deterministically re-score without needing private provider access to cached model outputs (the call_cache entries themselves contain the exact request/response fingerprints used in the run).

Practical tradeoffs and design guidance. Independent_per_condition sampling increases statistical informativeness but raises computational cost and exposure-management complexity. Multi-seed designs improve precision at the cost of increased model calls; the preregistered plan bounded calls conservatively (planned maximum_model_calls = 240) and specified stopping/retry rules; any rerun should adopt analogous resource bounds. Validator diversification improves construct validity but requires operational alignment: teams must decide whether the union (lenient) or intersection (conservative) of validators is the relevant deployment criterion. Importantly, all of these decisions are traceable and auditable using the provided manifest and SHA-256 list; reproducible governance is feasible by design.

Threats to validity (artifact-identified, expanded). Internal validity: the shared_initial design intentionally removed generation variance, producing a correct but uninformative confirmatory outcome. Construct validity: deterministic task templates and a single validator that enforces template constraints can inflate pass rates for canonical candidates; archived validator traces show that required_sequence checks were satisfied in all confirmed passes (see execution/scoring/task-000003-guarded.json SHA-256 dad1cf4cfd80b4cc...). External validity: the experiment used a single declared model (gpt-5-mini) and synthetic intent templates; results do not generalize to different LLM families, open natural-language intents, or production topologies. Conclusion validity: the degenerate contingency yields trivial hypothesis test outputs; interpreting $p = 1.0$ as evidence of equal operational reliability would be a category error. All these threats are documented and evidenced in the artifact bundle, enabling third parties to quantify them by rerunning the alternative designs described above.

Operational implications and recommended forensic checklist. For practitioners evaluating LLM→NetOps pipelines we offer an artifact-based checklist grounded in this study’s materials: (1) preregister the estimand and generation semantics (shared vs independent) and archive both; (2) publish canonical candidate files, provider call traces, and validator traces with SHA-256s to enable third-party audit; (3) when estimating generative robustness, use independent_per_condition sampling and multi-seed replication; (4) require validator diversification for high-stakes rollouts; (5) perform contamination/exposure checks over published candidates and provider traces (the bundle includes analysis/contamination_summary.csv for replication). Our run

demonstrates that following these steps makes both positive and null outcomes auditable and actionable.

Concluding interpretive note. The experiment as executed is scientifically honest, reproducible, and valuable: it demonstrates a fully-articulated pipeline and identifies a methodological pitfall that can silently convert an intended independent test into a deterministic sameness check. Because all artifacts and SHA-256 fingerprints (including the immutable initial master prompt provenance/master_prompt.txt SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15 and the preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdded71a3) are published, teams can (and should) deterministically transform this reproducible baseline into an informative assessment of independent generative reliability via the minimal, artifact-grounded reruns described above.

VI. LIMITATIONS

- Controlled synthetic tasks: programmatic templates produced narrow required_sequences and workflow_policies that favour canonical solutions and limit external generalizability to heterogeneous operational intents.
- Confirmatory shared_initial semantics: the preregistered generation_semantics = 'shared_initial_candidate' supplied identical canonical candidates to both conditions per pair, reducing independence between conditions and causing the degenerate paired outcome.
- Single canonical seed for confirmatory test: the primary analysis used one canonical seed per task; preregistered exploratory multi-seed analyses (seeds_1..4) were not included in the confirmatory inference reported here.
- Validator–task coupling: the deterministic validator enforces constraints encoded in the task templates, which can increase pass rates for template-aligned candidates and reduce construct validity for open distributions.
- Possible training-data exposure: preregistered contamination heuristics found no flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining.
- Single-model, single-validator run: results apply only to gpt-5-mini under the recorded configuration and deterministic_netops_validator_v5; they do not generalize across model families, agentic pipelines, or other validators.

VII. CONCLUSION

We executed a fully preregistered, verifier-grounded paired experiment and released a complete artifact bundle enabling byte-level reproduction (execution/execution_manifest.json and analysis/*). Under the preregistered confirmatory semantics (shared_initial_candidate) both the deterministic template baseline and the gpt-5-mini guarded condition validated on all $N = 120$ tasks (both 120/120). Archived evidence (shared_initial candidate files and validator scoring traces) demonstrates that identical canonical

candidates per pair and template-tight task constraints caused the degenerate paired outcome; consequently the confirmatory paired result does not provide evidence about independent LLM generative reliability in operational settings. We provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory execution, validator diversification, task heterogeneity expansion) that preserve reproducibility and enable informative independent comparisons. All execution and analysis artifacts—including the immutable master prompt reference (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba) and the preregistration SHA-256—are included in the submission bundle to permit exact audit and follow-up experiments. Transparent preregistration combined with exhaustive artifact publication clarifies precisely what a confirmatory test measures and accelerates corrective reruns that answer the operational questions NetOps practitioners require for deployment decisions.

DISCLOSURE STATEMENT

Mandatory disclosure (CNSM Agentic AI Researcher track). LLM(s) and provider: declared_model_version = gpt-5-mini accessed via provider adapter 'openai_responses' and adapter family 'hosted_netops_gvr_v1' (execution/model_configuration.json SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820).

Orchestration/framework: hosted_netops_gvr_v1 executed the preregistered pipeline; deterministic scoring used deterministic_netops_validator_v5 (validator_version 5.0). Preregistration: CAND-2026-001-NetConfig-Semantic-v1 (frozen prior to execution); preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdded71a37.

Exact initial master prompt: preserved as an immutable archived file provenance/master_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).

Human role and autonomy boundary: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the initial master prompt before the autonomy lock. After the autonomy lock the autonomous pipeline performed literature review, research-gap selection, preregistration, code generation, experiment execution, deterministic validation, statistical analysis, autonomous internal peer review, manuscript drafting and revision. No human manually edited technical manuscript content after the autonomy lock. Preregistered confirmatory generation semantics and consequence: confirmatory generation_semantics was set to 'shared_initial_candidate' so each pair received an identical canonical candidate for both baseline and guarded episodes; representative shared candidate artifacts: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...), execution/responses/task-000002-shared-initial.txt (SHA-256 ae967f70dfb6c...), execution/responses/task-000003-shared-initial.txt (SHA-256 f7f4db249ecbbce...). Validator and scoring: deterministic_netops_validator_v5 performed

normalized parsing, intent constraint checking, transient safety checks, and emitted per-episode JSON scoring traces archived under execution/scoring/*.json (representative SHA-256: execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...). Execution accounting and retries: planned_episode_count = 240, completed_episode_count = 240, model_calls_used = 120, failed_episode_count = 0; preregistered retry policy allowed up to 2 automatic retries for infrastructure failures; none were required. Contamination checks and reproducibility: preregistered string-overlap heuristics found no confirmatory flags under 2039b15ba holds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining. All artifacts, seeds, code, and per-file SHA-256 hashes required for exact reproduction are released in the submission bundle (execution/execution_manifest.json contains the exhaustive manifest). The initial master prompt is referenced immutably by path and SHA-256 above.

REFERENCES

- [1] <https://doi.org/10.1145/3603269.3604866>
- [2] <https://doi.org/10.23919/cnsm62983.2024.10814407>
- [3] <https://doi.org/10.1002/nem.70029>